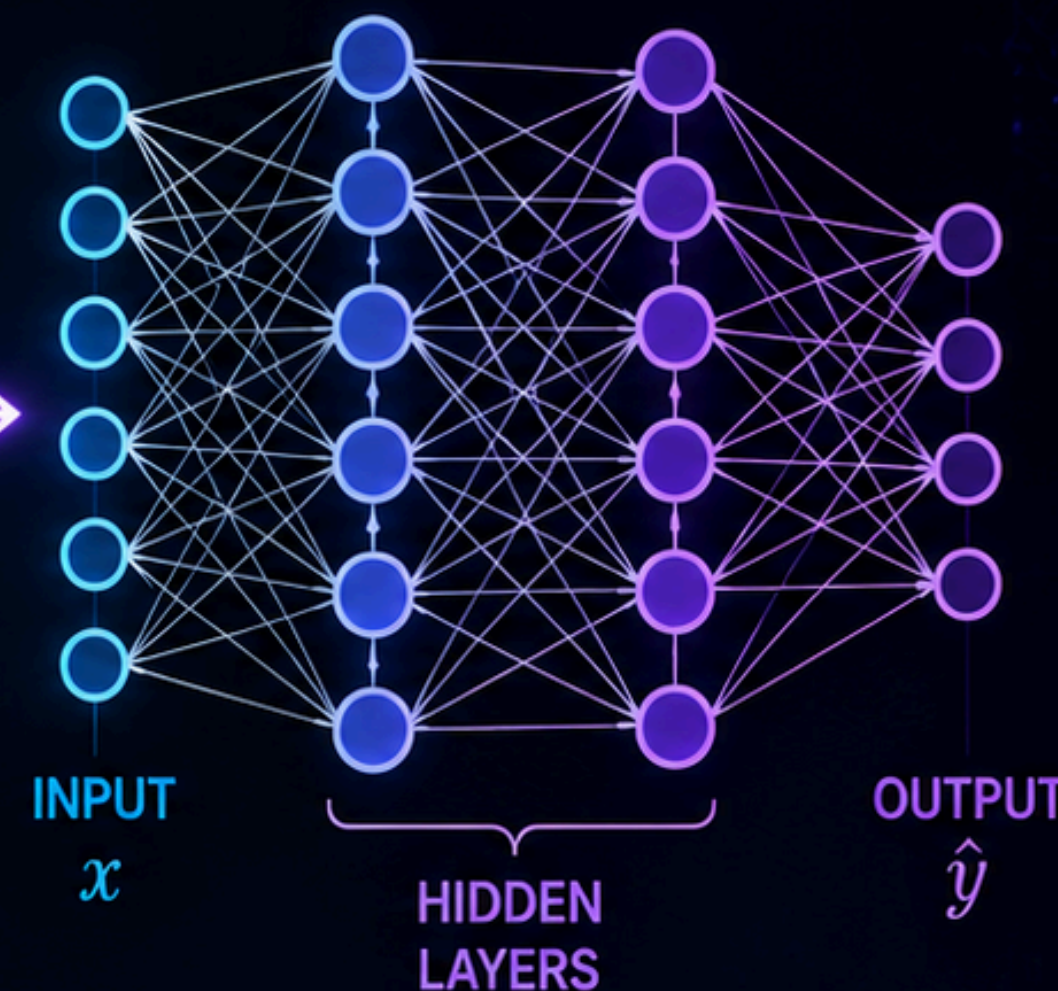


The Deep Learning Leap

Day 13:

Artificial Neural Networks and Loss

$$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$$
$$a^{[l]} = \sigma(z^{[l]})$$

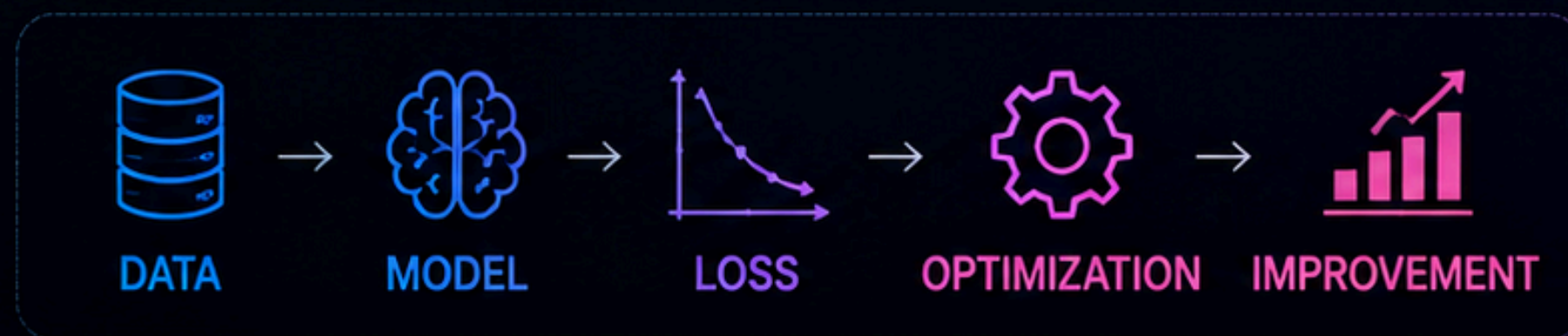
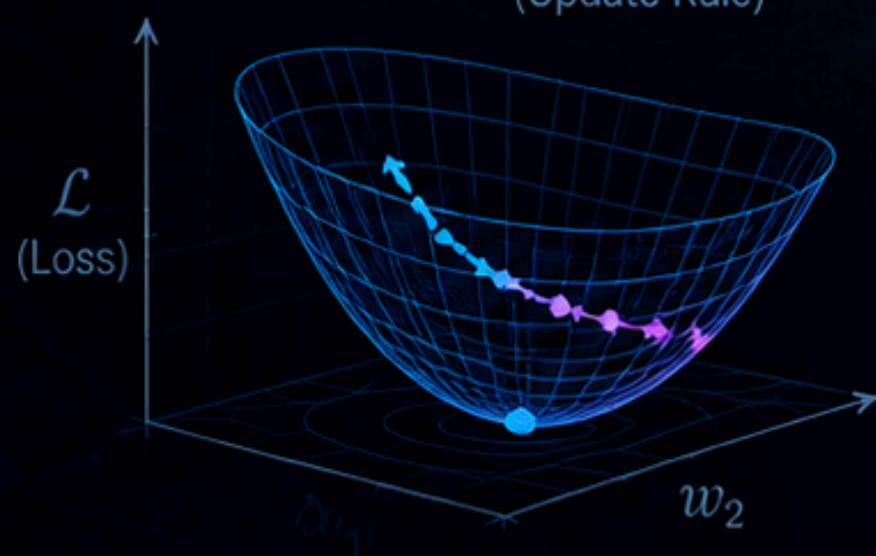


$$\frac{\partial \mathcal{L}}{\partial W^{[l]}}$$

(Gradient)

$$W^{[l]} := W^{[l]} - \alpha \frac{\partial \mathcal{L}}{\partial W^{[l]}}$$

(Update Rule)

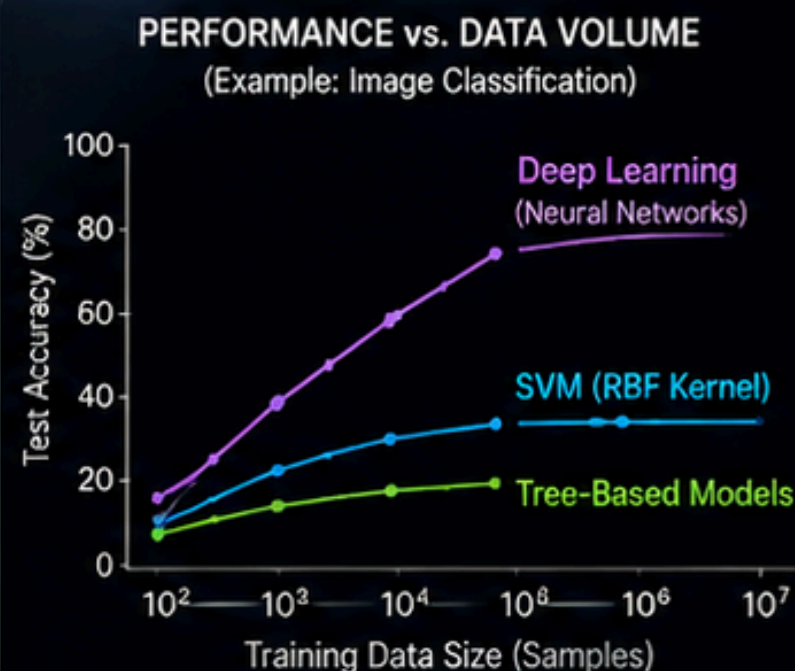
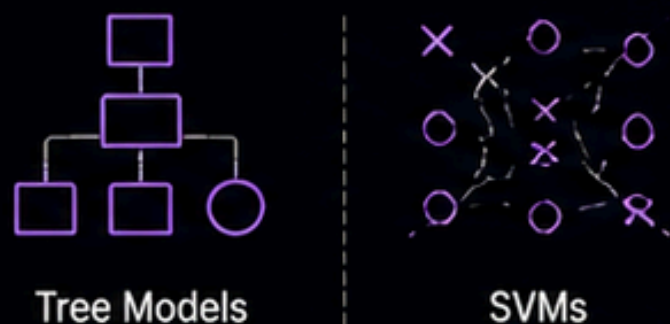


$$\mathcal{L}(\hat{y}, y) = \frac{1}{n} \sum_{i=1}^n \ell(\hat{y}_i, y_i)$$

↑ Loss ↑ Prediction ($\hat{y}$) ↑ True Label (y)

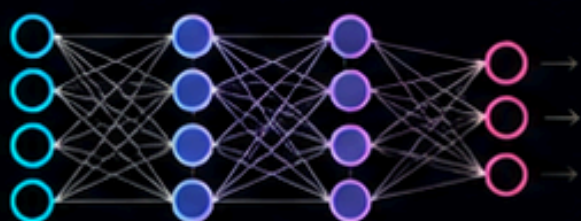
Why Deep Learning?

- 1 Tree models and SVMs plateau in performance as data volume increases.



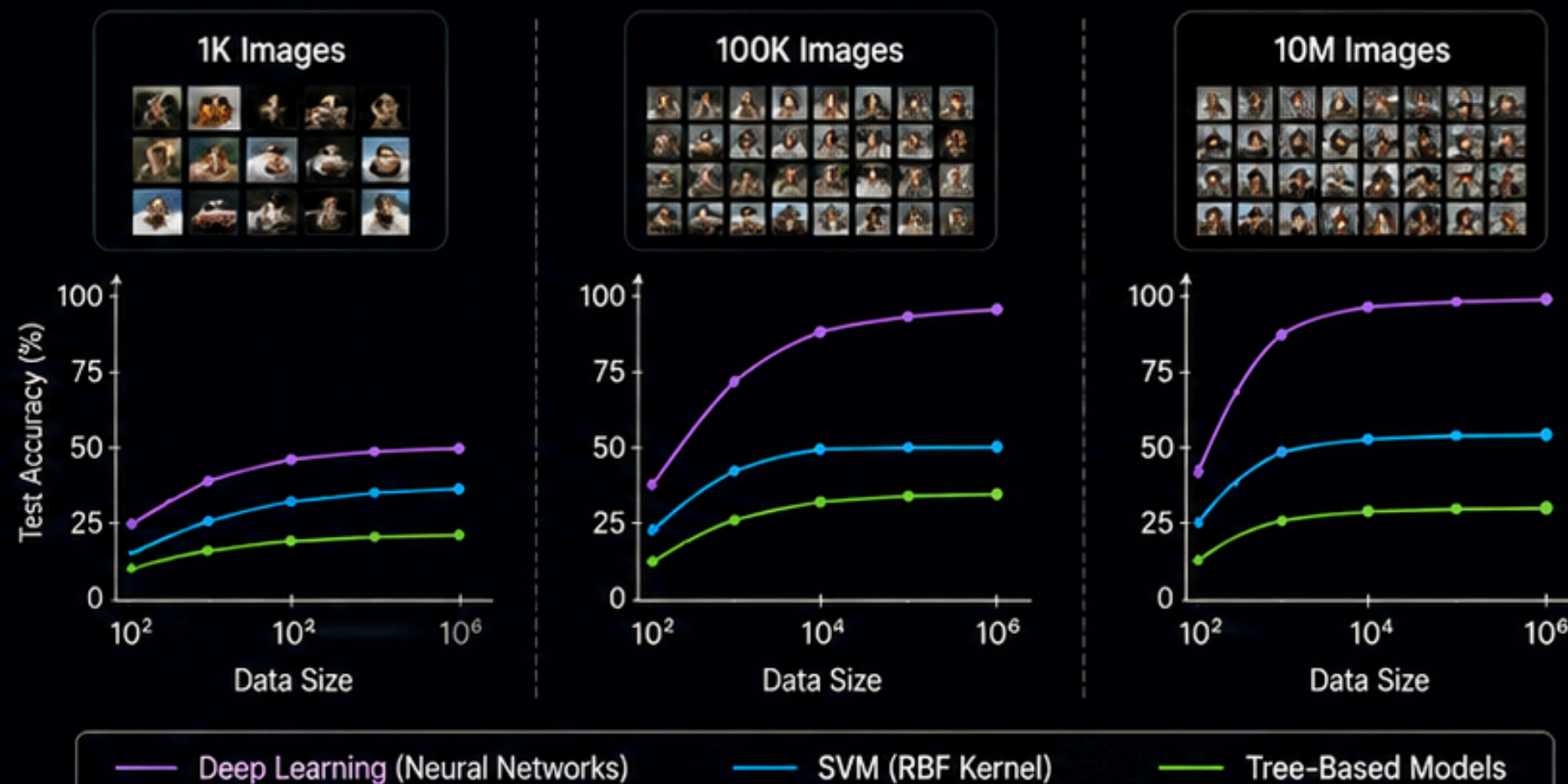
Traditional models hit a ceiling.
More data doesn't help much.

- 2 Neural Networks excel at extracting complex, non-linear features directly from raw inputs (pixels, text tokens) without manual feature engineering.



Neural Networks learn hierarchical representations automatically.

THE GAP WIDENS WITH MORE DATA



FROM RAW INPUT TO PREDICTION



Deep Learning scales with data and complexity—unlocking performance that traditional algorithms can't reach.

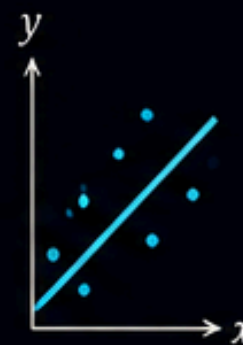
Activation Functions

Introducing Non-Linearity

1



=

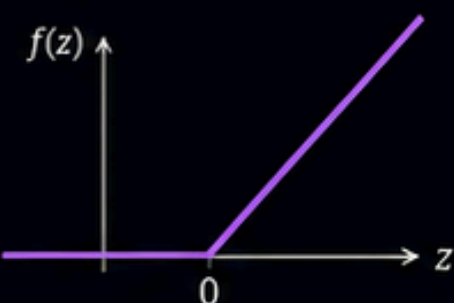


Without activation functions, a 100-layer neural network is mathematically identical to a single **linear regression line**.

2

ReLU

$$f(z) = \max(0, z)$$

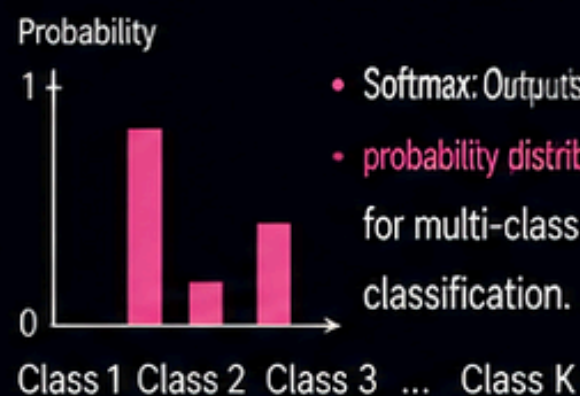


- **ReLU** (Rectified Linear Unit): Converts all negative numbers to zero.
- The **standard** for hidden layers.

3

Softmax

$$\sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

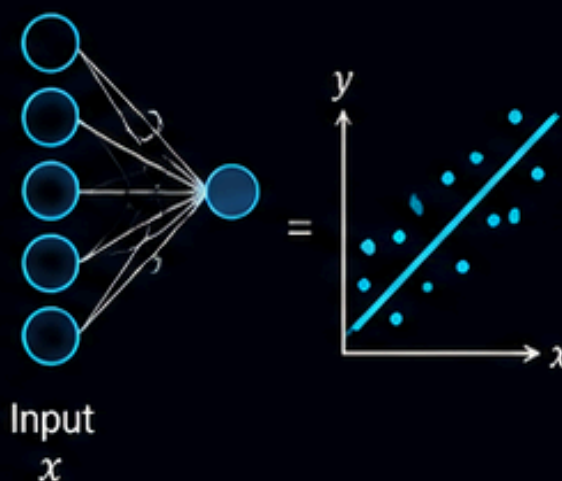


- Softmax: Outputs **probability distribution** for multi-class classification.

WHY NON-LINEARITY MATTERS

WITHOUT ACTIVATION (LINEAR NETWORK)

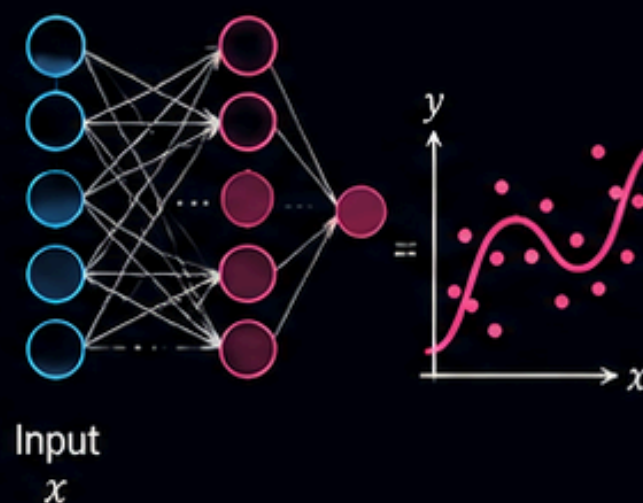
No matter how many layers we stack, the output is just a straight line.



100-Layer Linear Network
=
Single Linear Regression

WITH ACTIVATION (NON-LINEAR NETWORK)

Activation functions enable the network to learn complex, non-linear relationships.



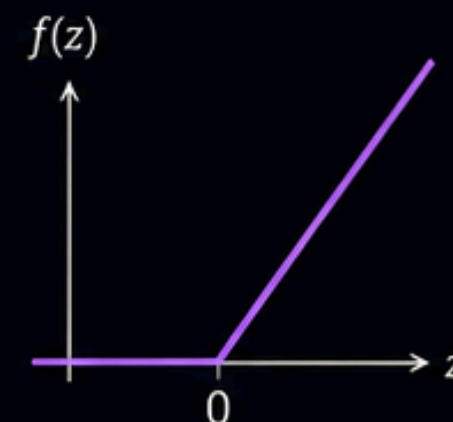
Deep Non-Linear Network
=
Complex Functions



Activation functions **break linearity** and give neural networks the power to **model complex patterns** in real-world data.

COMMON ACTIVATION FUNCTIONS

ReLU (Rectified Linear Unit)

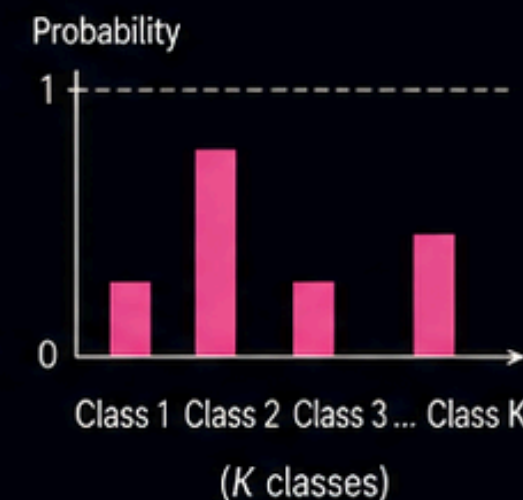


Properties:

- Simple and efficient
- Sparse activations
- Helps mitigate the vanishing gradient problem

Softmax

$$\sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$



Properties:

- Outputs in range $[0, 1]$
- Sum of all outputs = 1
- Ideal for multi-class classification



KEY TAKEAWAY:

Activation functions introduce the non-linearity needed for deep networks to learn complex patterns.

Loss Functions

Quantifying the Error



The **Loss Function** calculates the exact mathematical distance between the network's prediction and the true label.

True Label (y) Prediction ($\hat{y}$)



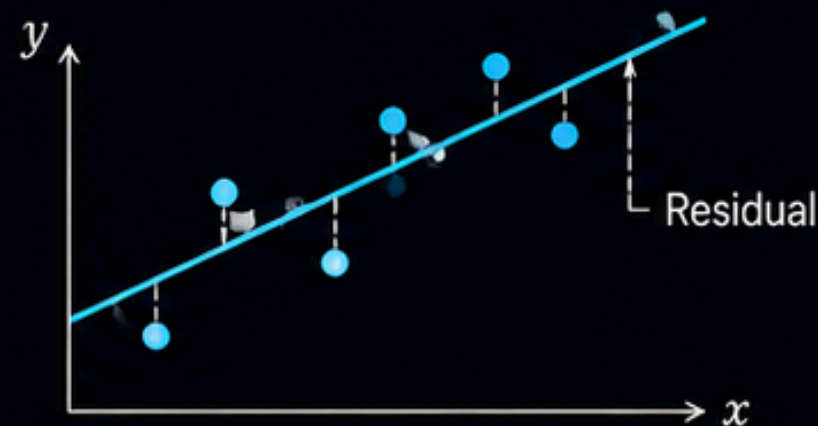
Error (Loss)
 L



Lower loss indicates predictions closer to the true labels.

1 Mean Squared Error (MSE)

Used for continuous regression tasks.



Formula

$$MSE = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

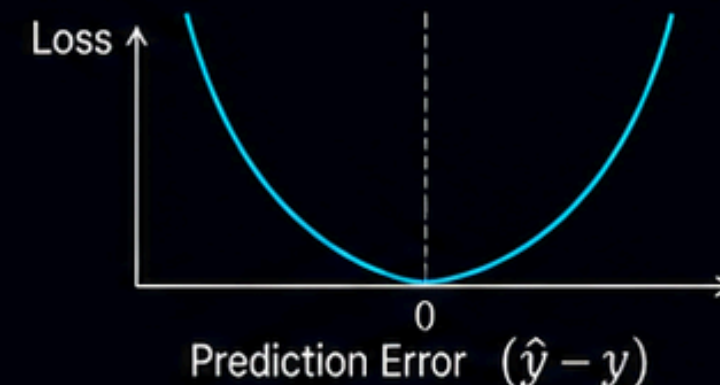
y_i = True value

$\hat{y}_i$ = Predicted value

n = Number of samples

How it works (Regression)

Squares the difference between prediction and true value. Penalizes larger errors more heavily.

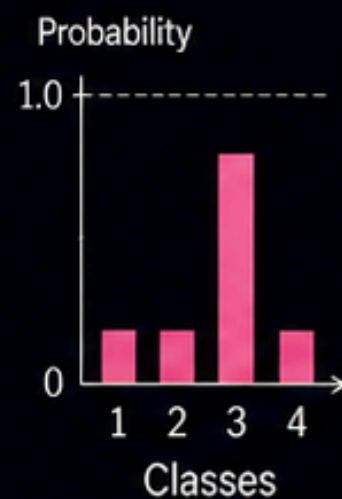


2 Categorical Cross-Entropy

Used for discrete classification tasks.

True Label (one-hot) Predicted Probabilities (Softmax Output)

0	Class 1	0.10
0	Class 2	0.15
1	Class 3	0.70
0	Class 4	0.05



Formula

$$CE = - \sum_{i=1}^K y_i \log(\hat{p}_i)$$

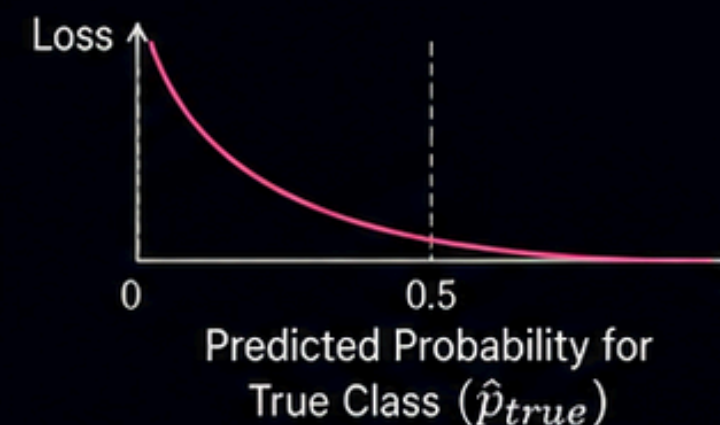
y_i = True label (0 or 1)

$\hat{p}_i$ = Predicted probability for class i

K = Number of classes

How it works (Classification)

Measures how well the predicted probability matches the true class. Heavily penalizes confident wrong predictions.



KEY TAKEAWAY: Loss functions turn prediction errors into a single number, guiding the network on how much to adjust its weights.

How the Network Learns

Optimization Intro

- 1 An initialized neural network makes completely **random guesses**.

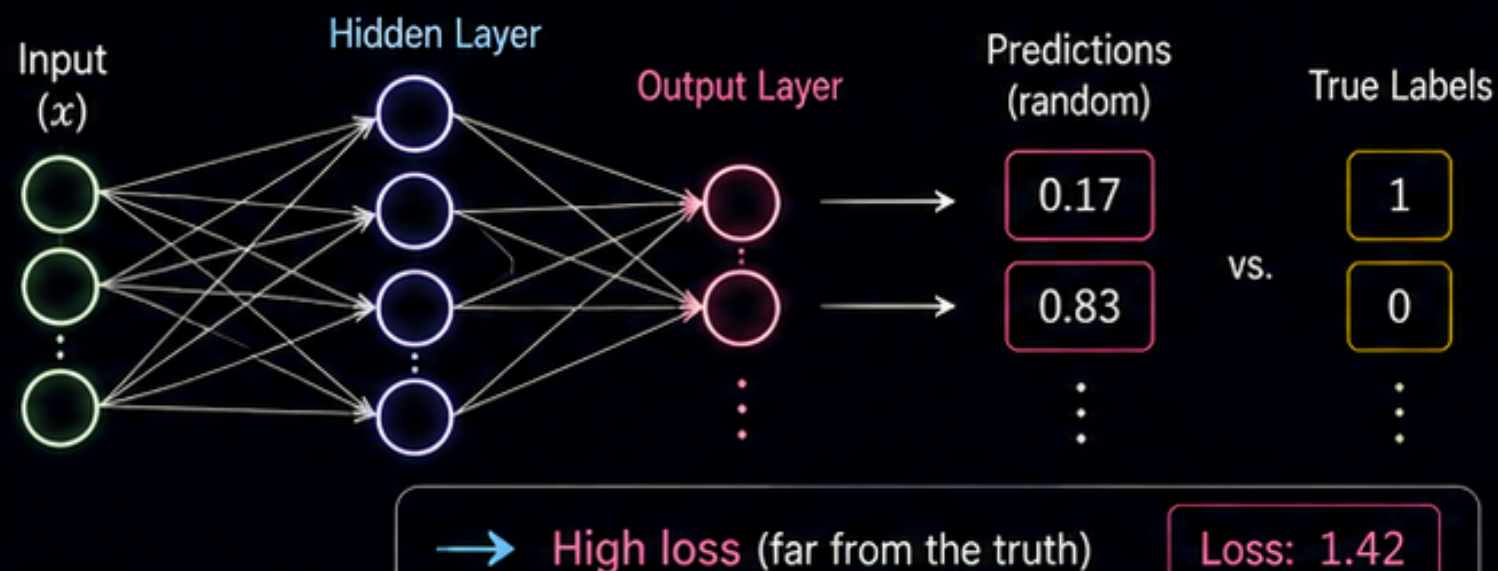


- 2 Optimization is the mathematical process of **updating the weights and biases** backwards through the network to **minimize the Loss**.



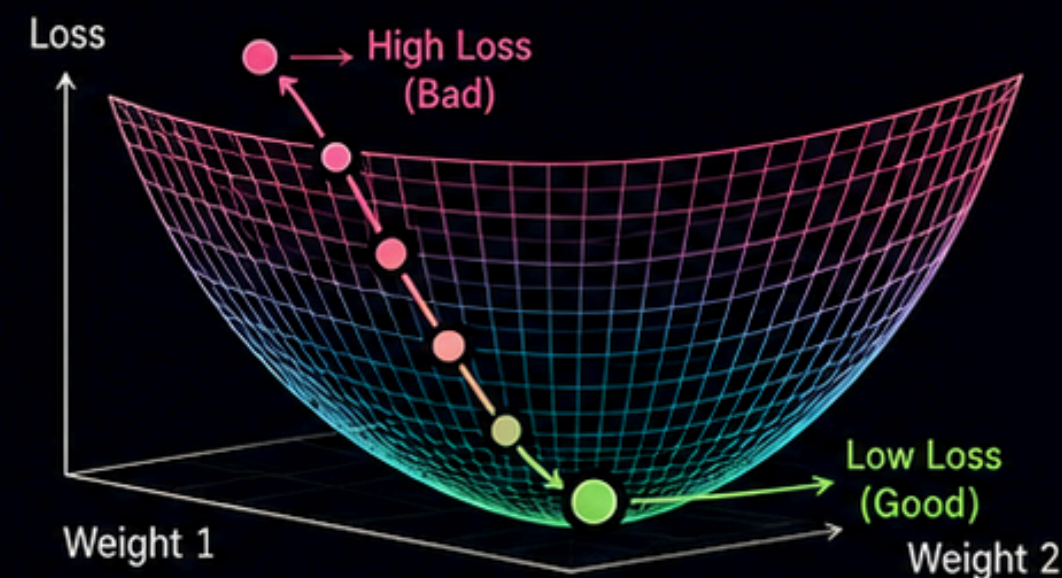
Goal: Find the parameters that produce the **lowest possible loss**.

1. INITIALIZED NETWORK: RANDOM GUESSES

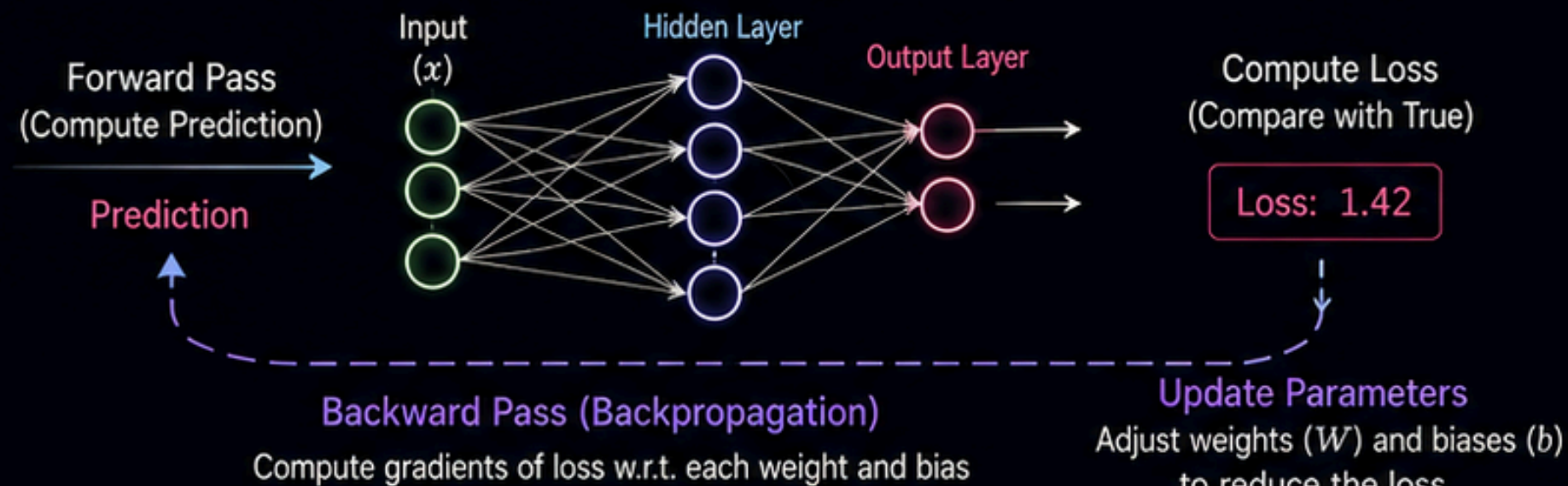


THE OPTIMIZATION GOAL

We want to move from a **high loss (bad)** to a **low loss (good)**.



2. OPTIMIZATION: UPDATE WEIGHTS & BIASES BACKWARDS



This repeats for many iterations...

Loss: 1.42
↓
Loss: 0.47
↓
Loss: 0.08
↓
Loss: 0.01

Loss
Decreases



KEY TAKEAWAY:

Optimization (via backpropagation + gradient-based methods) systematically adjusts the parameters so the network's predictions get closer and closer to the true labels.

Navigating the Loss Landscape

- 1** The algorithm calculates the **derivative (gradient)** of the Loss Function with respect to every single weight.

GRADIENT

$$\nabla_w L = \begin{bmatrix} \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \\ \vdots \\ \frac{\partial L}{\partial w_n} \end{bmatrix}$$

L = Loss
 w_i = a weight in the network
 $\frac{\partial L}{\partial w_i}$ = how much the loss changes with a small change in w_i

- 2** The gradient points in the direction of the **steepest ascent**; the network steps in the **opposite direction** to find the **lowest possible error**.

Update Rule

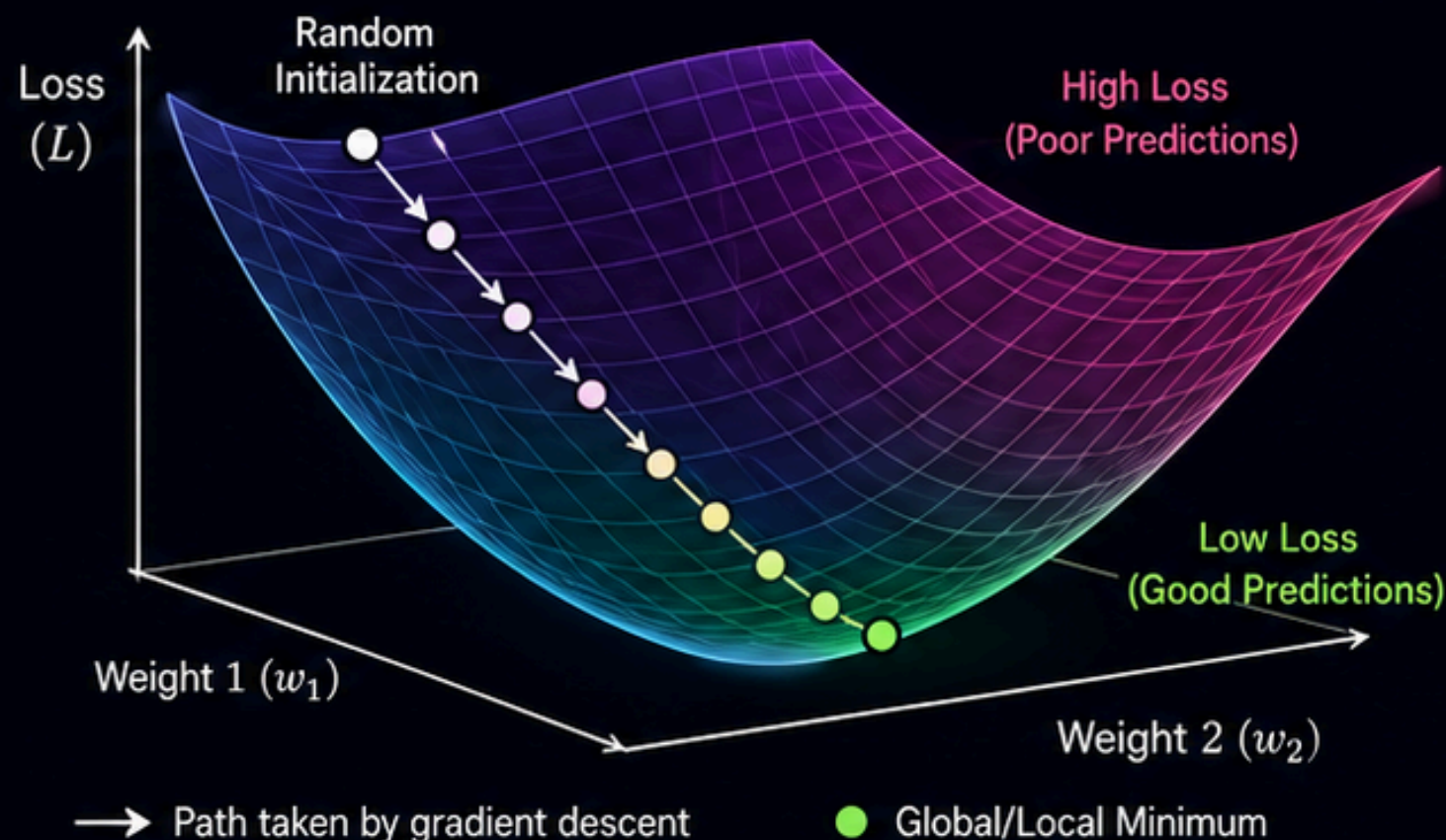
$$w_{\text{new}} = w_{\text{old}} - \eta \nabla_w L$$

η = Learning rate (step size)

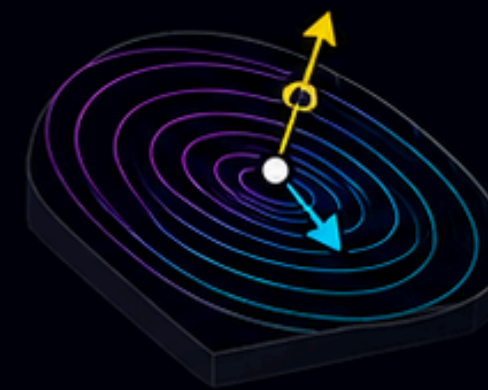
$\nabla_w L$ = Gradient

THE LOSS LANDSCAPE

A 2D slice of the high-dimensional loss surface.

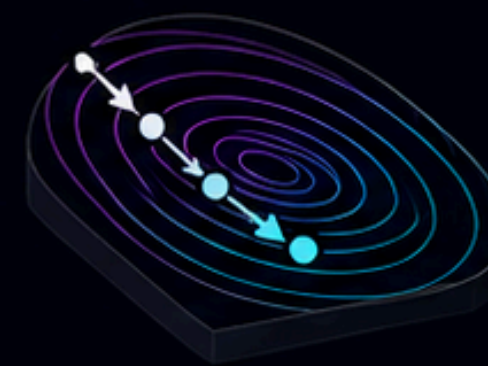


HOW THE GRADIENT WORKS



The gradient ($\nabla_w L$) points in the direction of the **steepest ascent** (greatest increase in loss).

→ Gradient (ascent)
 → Negative Gradient (descent)



By moving in the **opposite direction** ($-\nabla_w L$), the network moves downhill on the loss surface.

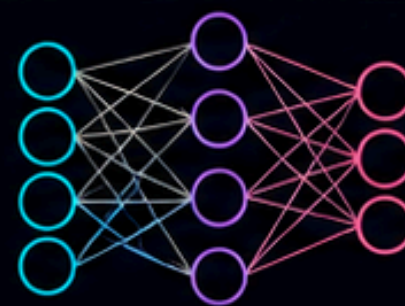
This continues iteratively until we reach a **minimum**.

GRADIENT FOR A NEURAL NETWORK

The gradient is computed for every parameter (weights and biases) in the network.

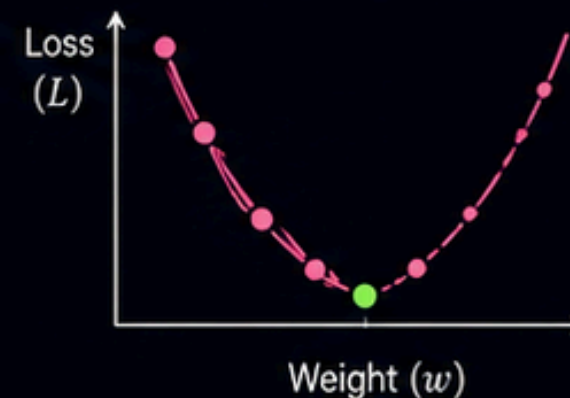
$$\nabla L = \left[\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \dots, \frac{\partial L}{\partial w_n}, \frac{\partial L}{\partial b_1}, \frac{\partial L}{\partial b_2}, \dots, \frac{\partial L}{\partial b_m} \right]$$

Where w_i are weights and b_j are biases.



All **weights** and **biases** are updated.

INTUITION IN 1D



1. Start at random weight
2. Compute gradient
3. Move opposite to gradient
4. Repeat until loss is minimized



KEY TAKEAWAY:

Gradient Descent is like finding the lowest point in a valley by always taking a step downhill. It's the core engine that drives learning in neural networks.



Controlling the Step Size

- 1 The **Learning Rate (α)** determines how aggressively the model updates its weights.



α = Learning Rate (Step Size)

- 2 If α is **too high**, the model **overshoots** the minimum and **diverges**.



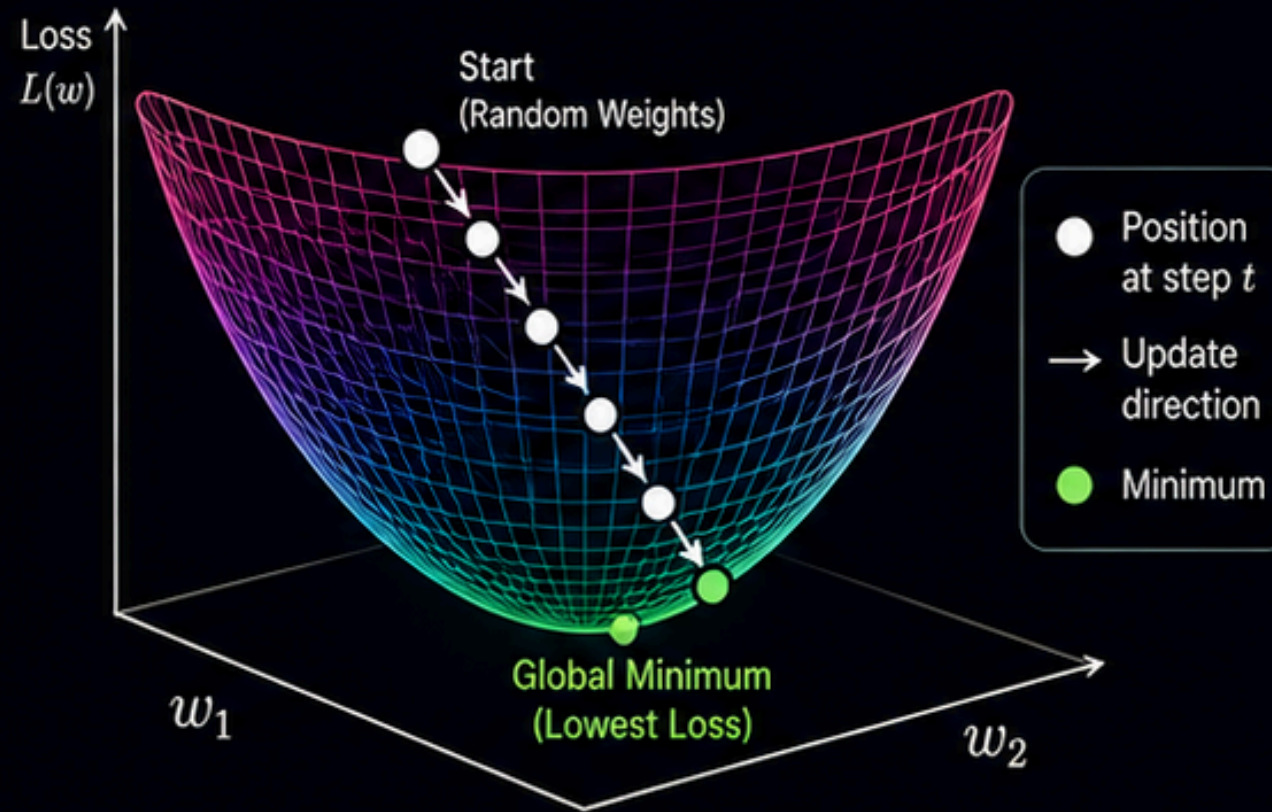
- 3 If α is **too low**, training takes **weeks**.



$$w_{new} = w_{old} - \alpha \nabla_w L$$

THE LOSS LANDSCAPE

Our goal is to reach the global minimum (lowest loss).

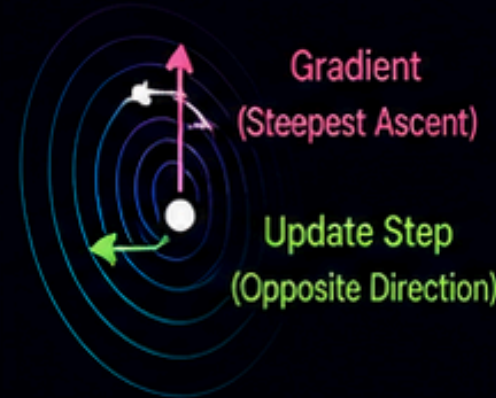


GRADIENT DESCENT UPDATE RULE

Weights are updated in the opposite direction of the gradient.

w = weights (parameters)
 $\nabla_w L$ = gradient of loss w.r.t. weights
 α = learning rate (step size)

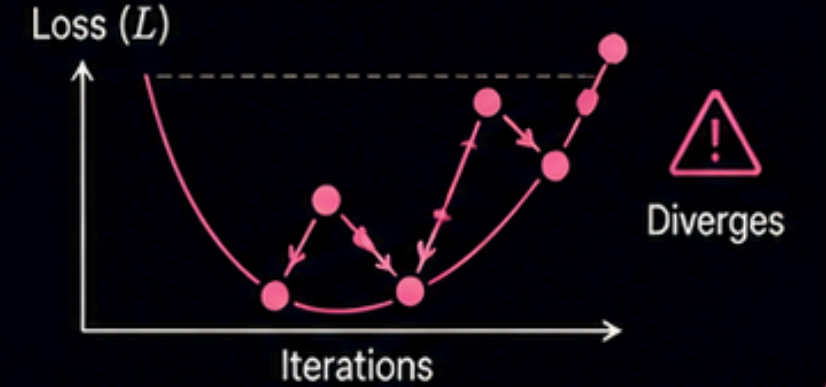
INTUITION



THE IMPACT OF LEARNING RATE (α)

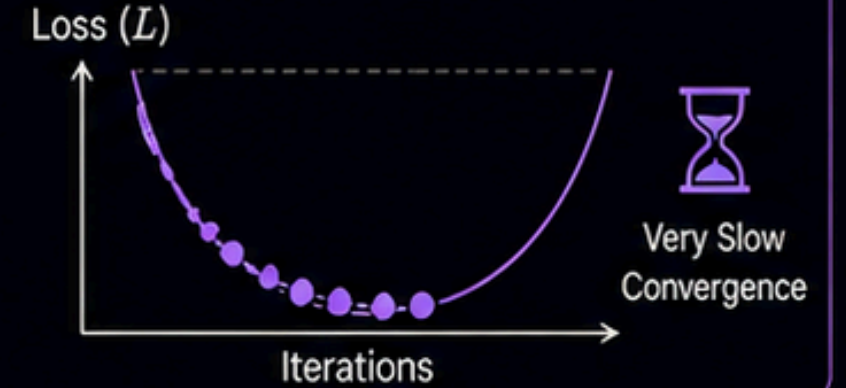
α Too High (Overshoot & Diverge)

Steps are too large. The model jumps over the minimum and diverges.



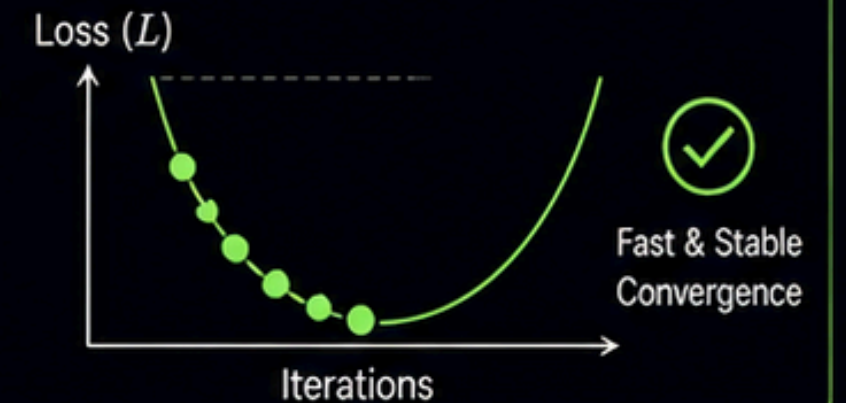
α Too Low (Too Slow)

Steps are too small. The model makes tiny progress and takes a very long time.



α Just Right (Efficient Convergence)

Steps are well-sized. The model reaches the minimum quickly and stably.



KEY TAKEAWAY:

The learning rate (α) controls the step size of each update. The right value is critical for fast, stable convergence to the lowest possible loss.

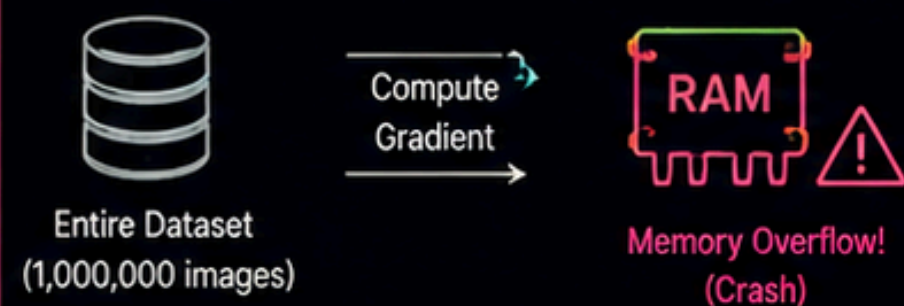


Stochastic Gradient Descent (SGD)

Beating the Memory Bottleneck

- 1 Calculating the gradient for an entire dataset of **1 million images at once** will **crash your RAM**.

Problem: Full-Batch Gradient Descent



- 2 SGD processes data in small, randomized "**Mini-Batches**" (e.g., 32 or 64 samples at a time), **updating the weights much faster**.

Solution: Stochastic Gradient Descent (SGD)

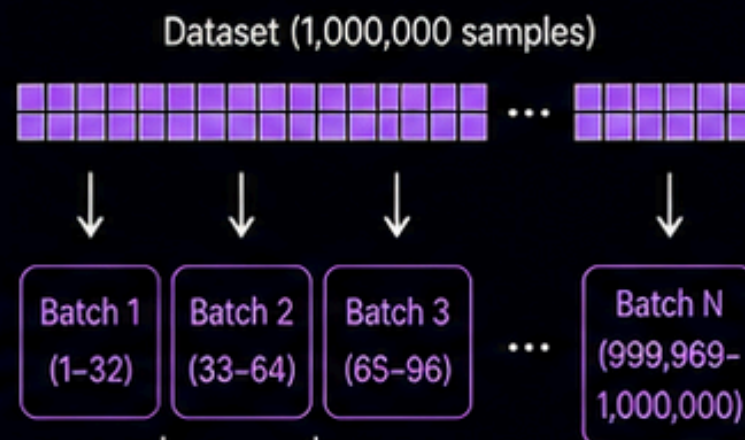


HOW SGD WORKS



MINI-BATCH EXAMPLE

Batch Size = 32



For each batch:

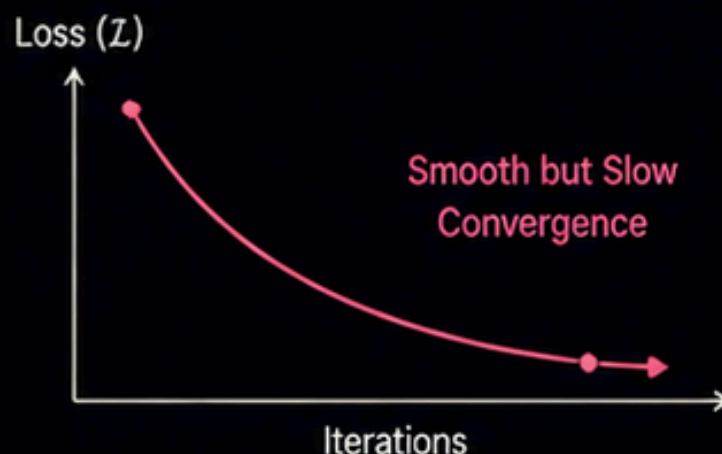
- Forward pass
- Compute loss
- Backward pass (compute gradients)
- Update weights

Repeat for all batches
(One Epoch Complete)

SGD vs. FULL-BATCH: THE TRADE-OFF

Full-Batch Gradient Descent

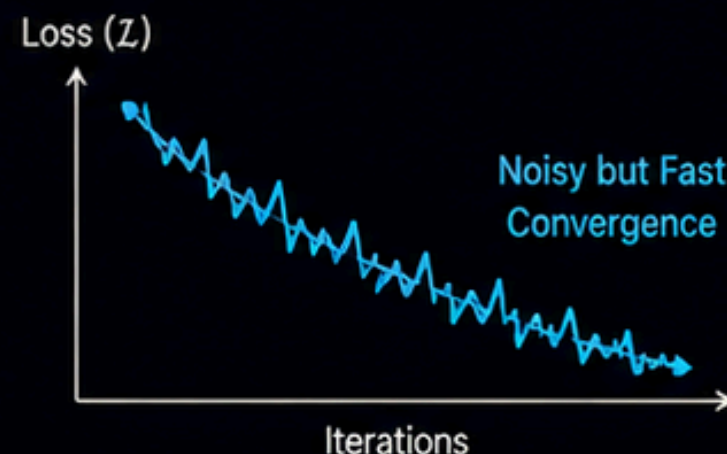
- + Stable and accurate gradient
- Very slow
- Extremely high memory usage



VS.

SGD (Mini-Batch Gradient Descent)

- + Very fast
- + Memory efficient
- Noisy but converges to minimum



KEY TAKEAWAY: SGD uses mini-batches to dramatically reduce memory usage and speed up training while still finding a good (near-optimal) solution.



Smaller batches = less memory, faster updates, noisier path, but great generalization!

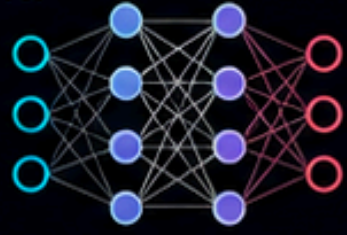
The Industry Standard

- 1 Writing backpropagation calculus from scratch in NumPy is **highly inefficient**.



Manual Backprop in NumPy
Slow, Error-Prone, Hard to Scale

VS.

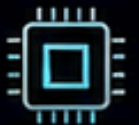


Frameworks
Fast, Reliable, Scalable

- 2 TensorFlow acts as the **high-performance C++ backend**.



High Performance



Optimized
Executions



GPU / TPU
Acceleration

- 3 Keras acts as the **Python-friendly API** that sits on top, allowing you to build deep architectures in **5 lines of code**.



```
Sequential([  
    ...  
])
```



Developer
Productivity

THE STACK: HOW IT ALL WORKS TOGETHER



Keras (Python API)

- High-level, easy-to-use
- Model building, training, evaluation
- Callbacks, metrics, visualizations



TensorFlow (C++ Backend)

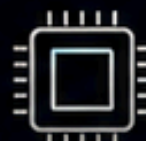
- Builds computation graphs
- Automatic differentiation (autodiff)
- Optimizes operations



Hardware



GPU



CPU



TPU



Result: Maximum Performance + Maximum Productivity

5 LINES OF KERAS CODE TO BUILD A DEEP MODEL



```
1 from tensorflow import keras  
2 model = keras.Sequential([  
3     keras.layers.Dense(128, activation='relu', input_shape=(784,)),  
4     keras.layers.Dense(10, activation='softmax')  
5 ])
```



Define
Architecture



Compile
Model



Train
Model



Evaluate
& Predict

WHY THIS MATTERS



Blazing Fast

Built-in optimizations
and hardware
acceleration.



Battle Tested

Used in production
at scale by thousands
of companies.



Developer Friendly

Focus on ideas,
not low-level
implementations.



KEY TAKEAWAY:

TensorFlow + Keras = The perfect combination of performance and productivity.
Focus on building amazing models, not re-inventing the math engine.



+

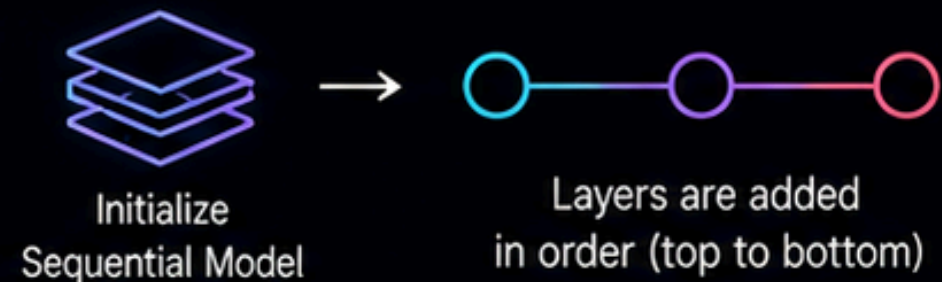


=



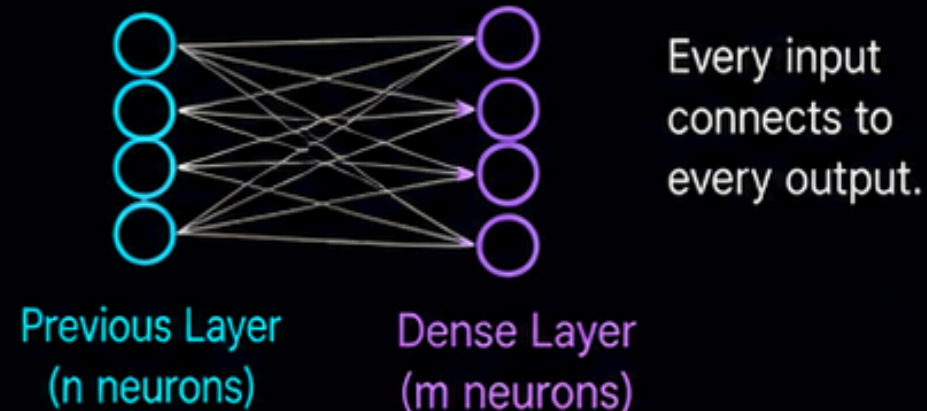
Stacking the Layers

- 1 **Sequential()** initializes a linear stack of layers.



```
model = Sequential()
```

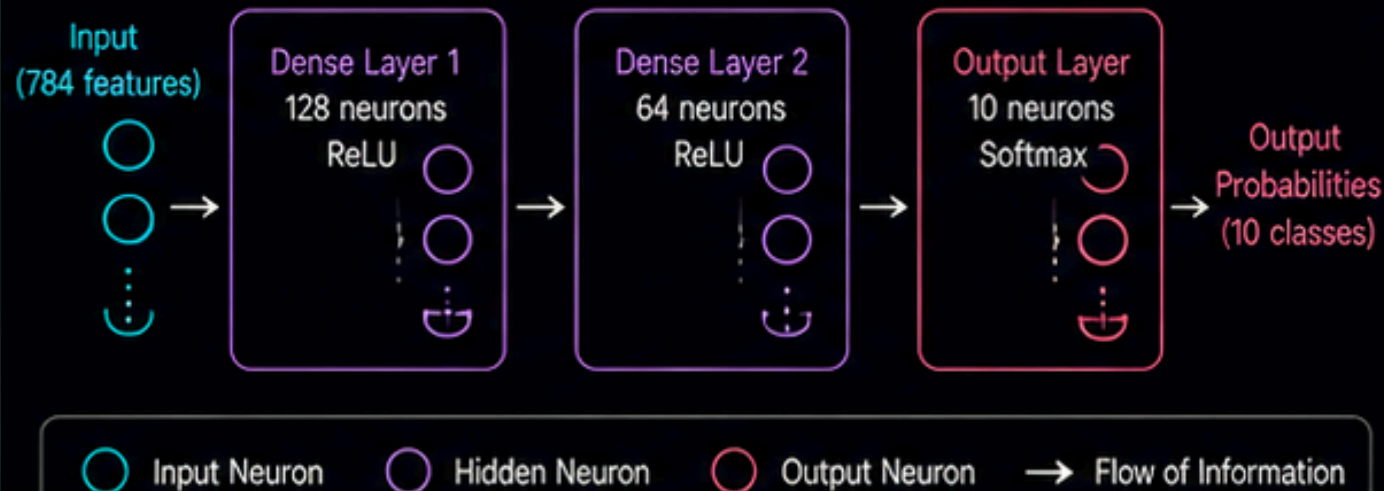
- 2 **Dense()** creates a fully connected hidden layer where every node connects to every node in the previous layer.



EXAMPLE: 3-LAYER SEQUENTIAL MODEL

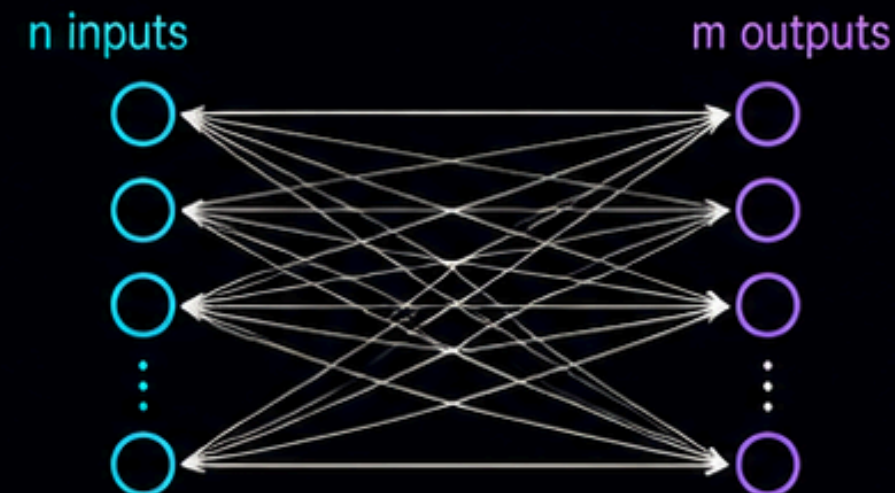
```
1 from tensorflow.keras.models import Sequential
2 from tensorflow.keras.layers import Dense
3
4 model = Sequential([
5     Dense(128, activation='relu', input_shape=(784,)),
5     Dense(64, activation='relu'),
6     Dense(10, activation='softmax')
7 ])
```

MODEL ARCHITECTURE (LINEAR STACK)



WHAT DOES DENSE() DO?

Creates a fully connected (affine) layer:



$$z = Wx + b$$

x : Input vector ($n \times 1$)
 W : Weight matrix ($m \times n$)
 b : Bias vector ($m \times 1$)
 z : Output before activation ($m \times 1$)

Activation(z) $\rightarrow$ Output

- ✓ Learns patterns by adjusting W and b
- ✓ More neurons = more capacity
- ✓ Stack layers = learn complex representations



KEY TAKEAWAY:

Sequential() stacks layers in order.

Dense() layers are fully connected and learn representations by adjusting weights and biases.



+



=



Preparing for Training

1 Before training, the model must be compiled with **three things**:

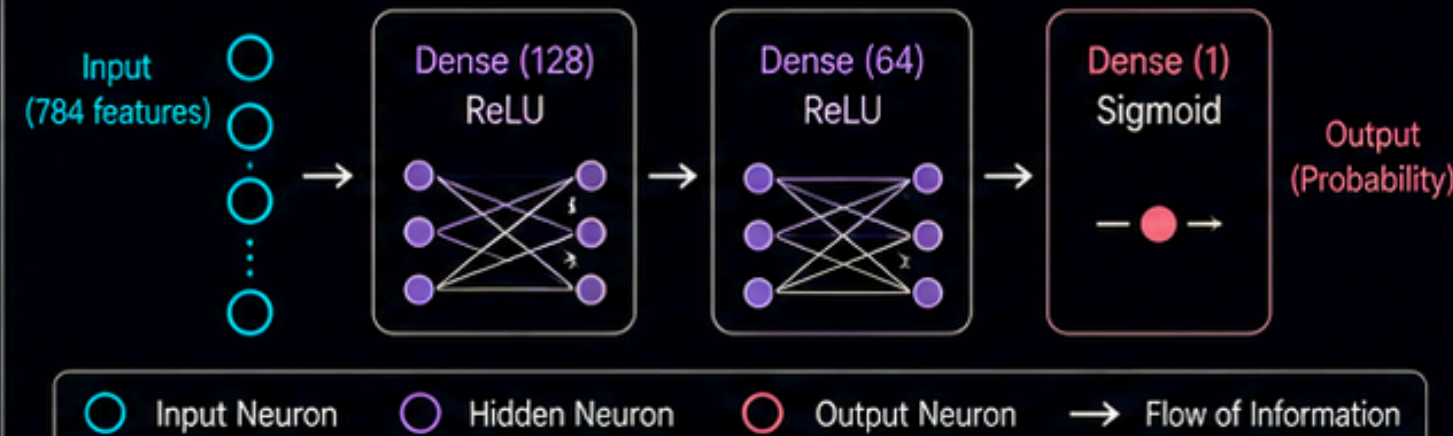
1  **The Optimizer**
Algorithm that updates the weights to minimize the loss.
(e.g., Adam or SGD)

2  **The Loss Function**
Measures how far the predictions are from the true labels.
(e.g., binary_crossentropy)

3  **The Metrics**
Used to evaluate performance during training and testing.
(e.g., accuracy)

 Compilation connects the model architecture with the training strategy.

RECAP: OUR SEQUENTIAL MODEL ARCHITECTURE



COMPILING THE MODEL

```
1 from tensorflow.keras.models import Sequential
2 from tensorflow.keras.layers import Dense
3
4 model = Sequential([
5     Dense(128, activation='relu', input_shape=(784,)),
6     Dense(64, activation='relu'),
7     Dense(1, activation='sigmoid')
8 ])
9
10 model.compile(
11     optimizer='adam',
12     loss='binary_crossentropy',
13     metrics=['accuracy']
14 )
```

Compile: Connects the architecture with the training configuration

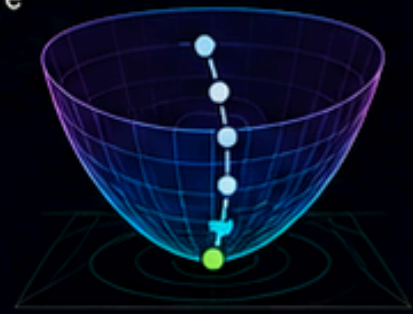
THE 3 COMPONENTS EXPLAINED

1  **OPTIMIZER**
Decides how the model's weights are updated based on the gradient.

Common Choices:

Adam

SGD



2  **LOSS FUNCTION**
Quantifies the error between the model's predictions and the true labels.

Common Choices:

binary_crossentropy

categorical_crossentropy

mse

$$\text{Loss} = -\frac{1}{N} \sum_{i=1}^N \left[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \right]$$

3  **METRICS**
Measures model performance. Reported during training and evaluation. (Does not affect learning.)

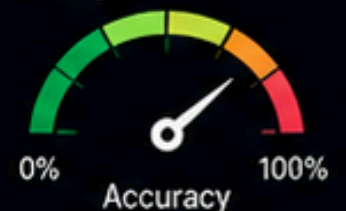
Common Choices:

accuracy

precision

recall

AUC



 **KEY TAKEAWAY:** Compilation is the bridge between your model and the training process. **Optimizer** minimizes the loss, **Loss** measures error, and **Metrics** track progress.


Model Architecture

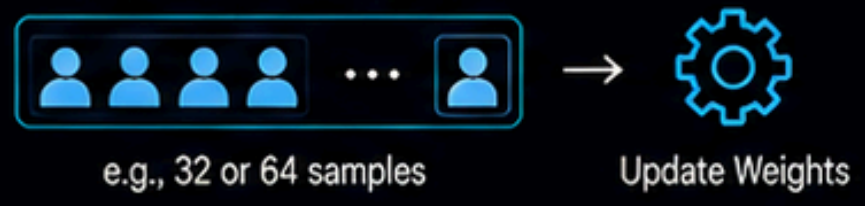
+ 
Compile()

= 
Ready to Train!

The Training Loop

1 Batch Size

The number of samples processed before the model updates its weights.



2 Epoch

One complete pass of the entire training dataset through the algorithm.



3 model.fit()

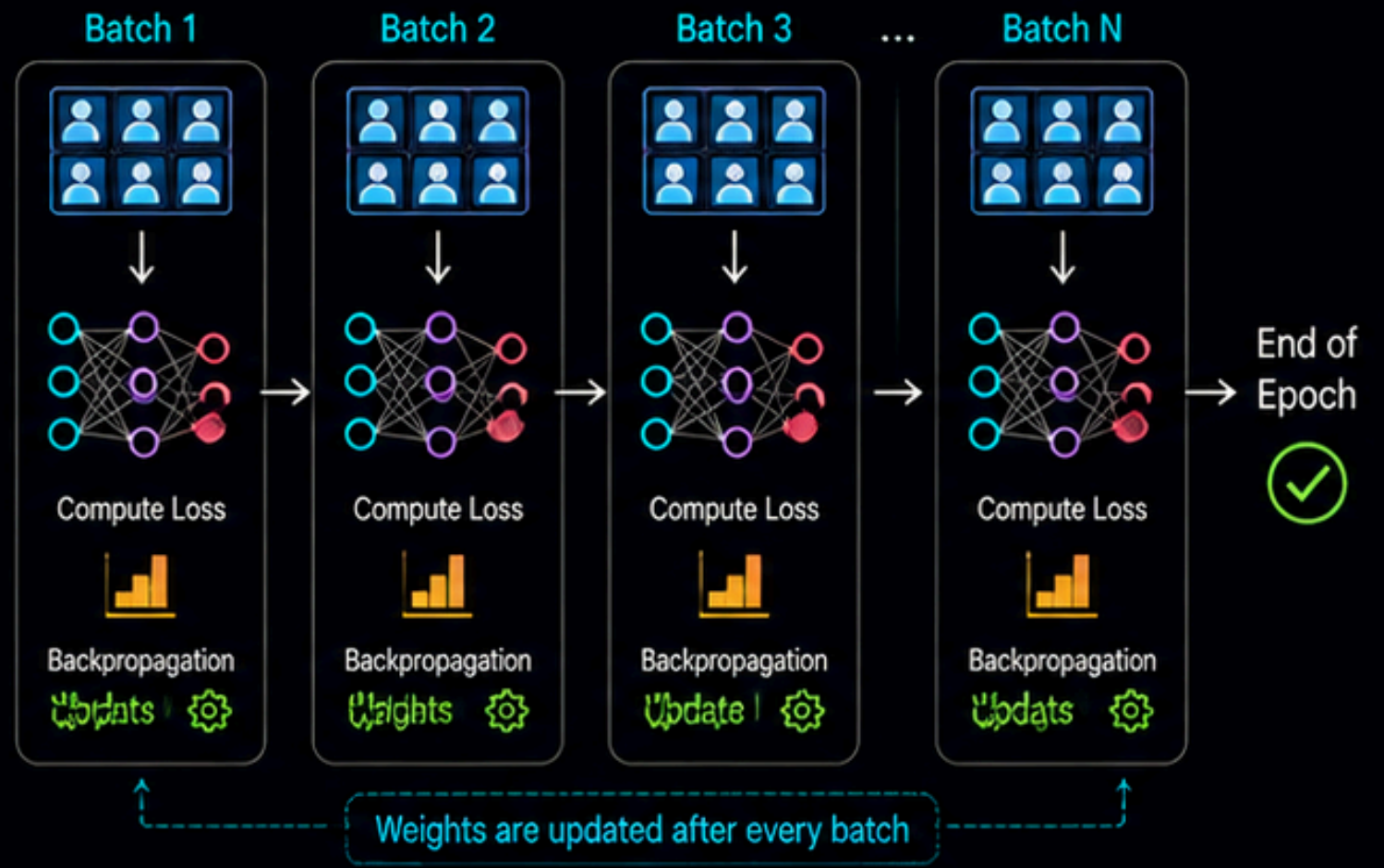
runs this entire loop automatically.



- ✓ Handles batching
- ✓ Shuffles data
- ✓ Updates weights
- ✓ Repeats for epochs

INSIDE ONE EPOCH

The training data is split into many mini-batches.



EXAMPLE: KERAS model.fit()

```
1 history = model.fit(  
2     X_train, y_train,      — Training data  
3     batch_size=32,        — Batch Size (e.g., 32)  
4     epochs=10,            — Number of Epochs  
5     shuffle=True,         — Shuffle each epoch  
6     validation_split=0.2  — Hold out for validation  
7 )
```

WHAT model.fit() DOES FOR YOU



WHY THIS MATTERS

- Efficient** Mini-batches use less memory and make training faster.
- Stable** Balance between noisy updates (small batches) and stable updates (large batches).
- Scalable** Works for small datasets and millions of samples.

THE FULL TRAINING LOOP OVER MULTIPLE EPOCHS



KEY TAKEAWAY:

- 1** Batch Size = samples per update.
- 2** Epoch = one full pass through the entire dataset.
- 3** model.fit() runs the whole training loop automatically.



The loop continues until the model learns the patterns in your data!